

BIBLE PYMUPDF (FITZ) – RÉFÉRENCE EXHAUSTIVE DU MODULE

LÉGENDE : [doc] = Document | [page] = Page | [rect] = Rect | [mat] = Matrix
[txt] = TextPage | [annot] = Annot | [draw] = DrawPath | [px] = Pixmap

[doc] Ouvrir et créer des documents – fitz.open()

DESCRIPTION : Charger un PDF existant ou créer un nouveau document.
POURQUOI : Point d'entrée unique pour tous les types de documents.
CONTEXTE : PDF, XPS, EPUB, MOBI, FB2, CBZ, SVG, images (PNG, JPEG...).

— OUVERTURE —

```
import fitz
```

doc = fitz.open("fichier.pdf")	Ouvre un PDF depuis un fichier
doc = fitz.open("fichier.pdf", password="secret")	Ouvre avec mot de passe
doc = fitz.open("image.png")	Ouvre une image comme document
doc = fitz.open("fichier.epub")	Ouvre un EPUB
doc = fitz.open()	Crée un document PDF vide
doc = fitz.open("pdf", data)	Ouvre depuis bytes en mémoire
doc = fitz.open(stream=bytes_data, filetype="pdf")	Ouvre depuis bytes (stream)
doc = fitz.open(stream=io.BytesIO(data), filetype="pdf")	Ouvre depuis BytesIO

— FORMATS SUPPORTÉS —

PDF, XPS, OXPS, CBZ, EPUB, MOBI, FB2, SVG	Documents
PNG, JPEG, BMP, GIF, TIFF, PNM, PGM, PPM	Images (en lecture)
HTML, XML (limité)	Web

— GESTIONNAIRE DE CONTEXTE —

```
with fitz.open("fichier.pdf") as doc:
    page = doc[0]
```

Ferme automatiquement

— ATTRIBUTS DU DOCUMENT —

doc.page_count	Nombre de pages
doc.name	Nom du fichier source
doc.is_pdf	True si PDF
doc.is_encrypted	True si chiffré
doc.needs_pass	True si mot de passe requis
doc.is_dirty	True si modifié non sauvegardé
doc.is_closed	True si fermé
doc.metadata	Dict des métadonnées
doc.permissions	Permissions du document
doc.language	Langue du document

— MÉTADONNÉES —

doc.metadata["title"]	Titre
doc.metadata["author"]	Auteur
doc.metadata["subject"]	Sujet
doc.metadata["keywords"]	Mots-clés
doc.metadata["creator"]	Application créatrice
doc.metadata["producer"]	Application productrice
doc.metadata["creationDate"]	Date de création
doc.metadata["modDate"]	Date de modification
doc.metadata["format"]	Format ("PDF 1.7"...)
doc.metadata["encryption"]	Type de chiffrement
doc.set_metadata(dict)	Modifie les métadonnées

EXEMPLES :

```
with fitz.open("rapport.pdf") as doc:
    print(f"{doc.page_count} pages")
    print(doc.metadata["title"])
```

[doc] Navigation et gestion des pages – doc[n] / load_page

DESCRIPTION : Accéder, insérer, supprimer et réorganiser les pages.
POURQUOI : Manipulation structurelle du document PDF.

— ACCÈS AUX PAGES —

<code>doc[0]</code>	Première page (index 0)
<code>doc[-1]</code>	Dernière page
<code>doc[n]</code>	Page n (0-indexé)
<code>doc.load_page(n)</code>	Alias de <code>doc[n]</code>
<code>for page in doc:</code>	Itère sur toutes les pages
<code>list(doc)</code>	Liste de toutes les pages
<code>doc.pages(start, stop, step)</code>	Itérateur avec slice

— GESTION DES PAGES —

<code>doc.insert_page(n)</code>	Insère une page vide en position n
<code>doc.insert_page(n, width=595, height=842)</code>	Avec dimensions (pts, A4 = 595x842)
<code>doc.new_page(pno=-1)</code>	Ajoute une nouvelle page à la fin
<code>doc.new_page(pno=-1, width=595, height=842)</code>	Avec dimensions
<code>doc.delete_page(n)</code>	Supprime la page n
<code>doc.delete_pages(from_page=2, to_page=5)</code>	Supprime un intervalle
<code>doc.delete_pages([0, 2, 4])</code>	Supprime des pages spécifiques
<code>doc.move_page(from_n, to_n)</code>	Déplace une page
<code>doc.copy_page(from_n, to_n)</code>	Copie une page
<code>doc.select([0, 2, 1])</code>	Réorganise / filtre les pages
<code>doc.select(range(doc.page_count-1, -1, -1))</code>	Inverse l'ordre des pages

— DIMENSIONS DE PAGE —

<code>page.rect</code>	Rectangle de la page (Rect)
<code>page.rect.width</code>	Largeur en points
<code>page.rect.height</code>	Hauteur en points
<code>page.mediabox</code>	MediaBox (boîte réelle)
<code>page.cropbox</code>	CropBox (zone visible)
<code>page.rotation</code>	Rotation (0, 90, 180, 270)
<code>page.set_rotation(90)</code>	Tourne la page
<code>page.number</code>	Numéro de la page (0-indexé)

EXEMPLES :

```
# Extraire pages paires uniquement
with fitz.open("doc.pdf") as doc:
    doc.select([i for i in range(doc.page_count) if i % 2 == 0])
    doc.save("pages_paires.pdf")

# Inverser les pages
doc.select(list(range(doc.page_count - 1, -1, -1)))
```

[page] Extraction de texte – `get_text()`

DESCRIPTION : Extraire le texte d'une page sous différents formats.

POURQUOI : Analyse de contenu, NLP, recherche, conversion.

CONTEXTE : Extraction simple à structurée avec positions.

— FORMATS D'EXTRACTION —

<code>page.get_text()</code>	Texte brut simple (défaut)
<code>page.get_text("text")</code>	Texte brut (alias)
<code>page.get_text("blocks")</code>	Blocs de texte avec positions
<code>page.get_text("words")</code>	Mots avec positions
<code>page.get_text("html")</code>	HTML avec mise en forme
<code>page.get_text("xhtml")</code>	XHTML
<code>page.get_text("xml")</code>	XML structuré
<code>page.get_text("json")</code>	JSON avec coordonnées complètes
<code>page.get_text("rawdict")</code>	Dict brut (très détaillé)
<code>page.get_text("dict")</code>	Dict structuré (spans, fonts...)
<code>page.get_text("rawjson")</code>	JSON brut
<code>page.get_text("page")</code>	Texte par page

— OPTIONS COMMUNES —

<code>page.get_text(flags=fitz.TEXT_PRESERVE_WHITESPACE)</code>	Conserve les espaces
<code>page.get_text(flags=fitz.TEXT_PRESERVE_IMAGES)</code>	Inclut les images inline
<code>page.get_text(flags=fitz.TEXT_INHIBIT_SPACES)</code>	Supprime les espaces auto
<code>page.get_text(flags=fitz.TEXT_DEHYPHENATE)</code>	Reconstruit les mots coupés
<code>page.get_text(clip=rect)</code>	Extrait seulement dans un rectangle
<code>page.get_text(sort=True)</code>	Trie le texte de haut en bas, gauche à droite

— FORMAT BLOCKS —

```
blocs = page.get_text("blocks")
# Chaque bloc : (x0, y0, x1, y1, texte, numéro_bloc, type)
# type 0 = texte, type 1 = image
for x0, y0, x1, y1, texte, n, type_ in blocs:
```

```

if type_ == 0:
    print(texte.strip())

```

```

— FORMAT WORDS
mots = page.get_text("words")
# Chaque mot : (x0, y0, x1, y1, mot, n_bloc, n_ligne, n_mot)
for x0, y0, x1, y1, mot, *_ in mots:
    print(f"{mot} @ ({x0:.1f}, {y0:.1f})")

```

```

— FORMAT DICT
data = page.get_text("dict")
for bloc in data["blocks"]:
    if bloc["type"] == 0:          # Texte
        for ligne in bloc["lines"]:
            for span in ligne["spans"]:
                print(span["text"])
                print(span["font"])    # Nom de la police
                print(span["size"])    # Taille en points
                print(span["flags"])   # Bold=16, Italic=2
                print(span["color"])   # Couleur (entier)
                print(span["bbox"])    # Coordonnées (x0,y0,x1,y1)
                print(span["origin"])  # Point d'origine du texte

```

```

EXAMPLES      :
# Texte complet d'un PDF
with fitz.open("doc.pdf") as doc:
    texte = "\n".join(page.get_text() for page in doc)

# Texte avec position, trié
mots = page.get_text("words", sort=True)
texte = " ".join(m[4] for m in mots)

# Détecter le gras
data = page.get_text("dict")
for bloc in data["blocks"]:
    if bloc.get("type") == 0:
        for ligne in bloc["lines"]:
            for span in ligne["spans"]:
                if span["flags"] & 16:    # Bit 4 = gras
                    print(f"GRAS: {span['text']}")

```

[page] Recherche dans le texte – search_for / get_text_selection

DESCRIPTION : Chercher du texte et obtenir ses coordonnées sur la page.
 POURQUOI : Localiser des mots, surligner, annoter, remplacer.

FONCTIONS PRINCIPALES :

page.search_for("texte")	Liste de Rect (zones du texte)
page.search_for("texte", quads=True)	Résultats en Quad (quadrilatères)
page.search_for("texte", flags=fitz.TEXT_DEHYPHENATE)	Insensible à la casse
page.search_for("texte", clip=rect)	Dans une zone définie
# Accès via document	
doc.get_page_text(n)	Texte brut de la page n
doc.search_page_for(n, "texte")	Cherche dans la page n

```

EXAMPLES      :
rects = page.search_for("Total")
for r in rects:
    print(f"Trouvé à : {r}")
# Surligner
annot = page.add_highlight_annot(r)

```

[px] Rendu en image – get_pixmap / Pixmap

DESCRIPTION : Convertir une page PDF en image bitmap (PNG, JPEG, etc.).
 POURQUOI : Prévisualisation, OCR, export image, vignettes.
 CONTEXTE : Conversion PDF→image, preprocessing pour ML/OCR.

— **RENDU D'UNE PAGE** —

<code>mat = fitz.Matrix(2, 2)</code>	Facteur de zoom (2x = 144 DPI)
<code>mat = fitz.Matrix(300/72, 300/72)</code>	300 DPI (72 = résolution native)
<code>pix = page.get_pixmap()</code>	Pixmap à 72 DPI (défaut)
<code>pix = page.get_pixmap(matrix=mat)</code>	Avec zoom/DPI
<code>pix = page.get_pixmap(dpi=150)</code>	DPI direct (PyMuPDF 1.19+)
<code>pix = page.get_pixmap(alpha=False)</code>	Sans canal alpha (plus petit)
<code>pix = page.get_pixmap(colorspace=fitz.csGRAY)</code>	En niveaux de gris
<code>pix = page.get_pixmap(colorspace=fitz.csRGB)</code>	En RGB (défaut)
<code>pix = page.get_pixmap(clip=rect)</code>	Rend seulement une zone
<code>pix = page.get_pixmap(annots=False)</code>	Sans les annotations

— **PIXMAP – ATTRIBUTS** —

<code>pix.width</code>	Largeur en pixels
<code>pix.height</code>	Hauteur en pixels
<code>pix.n</code>	Nombre de composantes (3=RGB, 4=RGBA)
<code>pix.colorspace</code>	Espace colorimétrique
<code>pix.xres</code>	Résolution horizontale (DPI)
<code>pix.yres</code>	Résolution verticale (DPI)
<code>pix.samples</code>	Données brutes (bytes)
<code>pix.stride</code>	Longueur d'une ligne (bytes)
<code>pix.size</code>	Taille totale (bytes)
<code>pix.irect</code>	Rectangle en pixels entiers
<code>pix.is_monochrome</code>	True si monochrome

— **SAUVEGARDE ET EXPORT** —

<code>pix.save("page.png")</code>	Sauvegarde en PNG
<code>pix.save("page.jpg")</code>	Sauvegarde en JPEG
<code>pix.save("page.pam")</code>	Sauvegarde en PAM
<code>pix.pil_image()</code>	Convertit en image Pillow (PIL)
<code>pix.pil_save("page.png")</code>	Sauvegarde via Pillow
<code>pix.tobytes()</code>	Retourne les bytes PNG
<code>pix.tobytes("jpeg", jpg_quality=90)</code>	Bytes JPEG
<code>pix.tobytes("png")</code>	Bytes PNG
<code>numpy_array = pix.samples_mv</code>	→ ndarray via memoryview + numpy

— **VERS NUMPY / PILLOW** —

```
import numpy as np
arr = np.frombuffer(pix.samples, dtype=np.uint8)
arr = arr.reshape(pix.height, pix.width, pix.n)

from PIL import Image
img = pix.pil_image()
# ou :
img = Image.frombytes("RGB", [pix.width, pix.height], pix.samples)
```

EXEMPLES :

```
# Convertir toutes les pages en PNG
with fitz.open("doc.pdf") as doc:
    for i, page in enumerate(doc):
        pix = page.get_pixmap(dpi=150)
        pix.save(f"page_{i:03d}.png")

# Vignette 200px de large
page = doc[0]
scale = 200 / page.rect.width
mat = fitz.Matrix(scale, scale)
pix = page.get_pixmap(matrix=mat)
pix.save("vignette.png")
```

[page] Extraction d'images – <code>get_images / extract_image</code>
--

DESCRIPTION : Extraire les images embarquées dans un PDF.
POURQUOI : Récupérer les illustrations, logos, photos d'un PDF.

FONCTIONS PRINCIPALES :

<code>page.get_images()</code>	Liste d'infos images de la page
<code>page.get_images(full=True)</code>	Infos complètes
<code>doc.get_page_images(n)</code>	Images de la page n
<code>doc.extract_image(xref)</code>	Extrait une image par son xref

— **STRUCTURE `get_images()`** —

```
# (xref, smask, width, height, bpc, colorspace, alt_colorspace, name, filter, referencer)
```

```

images = page.get_images()
for img in images:
    xref = img[0]
    info = doc.extract_image(xref)
    # info["image"]      → bytes de l'image
    # info["ext"]        → extension ("png", "jpeg"...)
    # info["width"]      → largeur
    # info["height"]     → hauteur
    # info["colorspace"] → espace colorimétrique

```

```

EXEMPLES :
with fitz.open("doc.pdf") as doc:
    for i, page in enumerate(doc):
        for j, img in enumerate(page.get_images()):
            xref = img[0]
            data = doc.extract_image(xref)
            ext = data["ext"]
            with open(f"img_p{i}_n{j}.{ext}", "wb") as f:
                f.write(data["image"])

```

[page] Extraction de tableaux – find_tables |

DESCRIPTION : Détecter et extraire les tableaux d'une page PDF.
 POURQUOI : Données tabulaires sans conversion manuelle.
 CONTEXTE : Rapports financiers, factures, données structurées. (PyMuPDF 1.23+)

FONCTIONS PRINCIPALES :

tabs = page.find_tables()	Détecte tous les tableaux
tabs = page.find_tables(clip=rect)	Dans une zone
tabs = page.find_tables(strategy="lines")	Stratégie de détection
tabs = page.find_tables(strategy="lines_strict")	
tabs = page.find_tables(strategy="text")	
len(tabs.tables)	Nombre de tableaux trouvés
tab = tabs[0]	Premier tableau
tab.extract()	→ liste de listes (str)
tab.to_pandas()	→ DataFrame pandas (si pandas installé)
tab.bbox	Rectangle du tableau
tab.col_count	Nombre de colonnes
tab.row_count	Nombre de lignes
tab.rows	Liste des lignes
tab.header	Ligne d'en-tête
tab.header.names	Liste des noms de colonnes

```

EXEMPLES :
page = doc[0]
tabs = page.find_tables()
if tabs.tables:
    df = tabs[0].to_pandas()
    print(df)

# Extraction manuelle
for row in tabs[0].extract():
    print(row) # ["Col1", "Col2", "Col3"]

```

[page] Annotations – add_*_annot |

DESCRIPTION : Ajouter, lire et modifier des annotations sur les pages.
 POURQUOI : Surlignage, commentaires, signatures, tampons.

— AJOUT D'ANNOTATIONS —

page.add_highlight_annot(rect)	Surlignage jaune
page.add_highlight_annot(quads)	Surlignage sur quadrilatères
page.add_underline_annot(rect)	Soulignement
page.add_strikeout_annot(rect)	Barré
page.add_squiggly_annot(rect)	Ondulé
page.add_text_annot(point, "commentaire")	Note collante
page.add_freetext_annot(rect, "texte")	Texte libre
page.add_rect_annot(rect)	Rectangle
page.add_circle_annot(rect)	Cercle/Ellipse

page.add_line_annot(p1, p2)	Ligne entre deux points
page.add_polyline_annot(points)	Polyligne
page.add_polygon_annot(points)	Polygone
page.add_stamp_annot(rect, stamp=0)	Tampon prédéfini
page.add_redact_annot(rect)	Zone de caviardage
page.add_link(dict)	Lien hypertexte
page.add_ink_annot(list_of_points)	Encre libre

— LIRE LES ANNOTATIONS —

for annot in page.annots():	Itère toutes les annotations
for annot in page.annots(types=[fitz.PDF_ANNOT_HIGHLIGHT]):	Filtrées
annot.type	Type de l'annotation
annot.rect	Rectangle
annot.info	Dict d'infos (auteur, date, contenu)
annot.info["content"]	Texte du commentaire
annot.info["title"]	Auteur
annot.colors	Couleurs (stroke, fill)
annot.vertices	Sommets (quadrilatères, polygones)

— MODIFIER ET SUPPRIMER —

annot.set_info(content="Nouveau commentaire")	
annot.set_colors(stroke=(1,0,0))	Rouge
annot.set_colors(fill=(1,1,0))	Jaune
annot.set_opacity(0.5)	Opacité
page.delete_annot(annot)	Supprime l'annotation

— CAVIARDAGE (REDACTION) —

```

rects = page.search_for("confidentiel")
for r in rects:
    page.add_redact_annot(r)
page.apply_redactions()
page.apply_redactions(images=fitz.PDF_REDACT_IMAGE_NONE)

```

Applique (irréversible)
Sans toucher aux images

EXEMPLES :

```

# Surligner toutes les occurrences d'un mot
rects = page.search_for("important")
for r in rects:
    hl = page.add_highlight_annot(r)
    hl.set_colors(stroke=(1, 1, 0)) # Jaune
    hl.update()

# Annoter avec un commentaire
point = fitz.Point(100, 100)
annot = page.add_text_annot(point, "À revoir")
annot.set_info(title="Reviewer")
annot.update()

```

[page] Dessin et insertion de contenu – draw / insert

DESCRIPTION : Ajouter formes, texte, images sur une page.
POURQUOI : Créer des PDF, apposer des tampons, watermarks, formulaires.

— FORMES —

shape = page.new_shape()	Commence un groupe de formes
shape.draw_line(p1, p2)	Ligne
shape.draw_rect(rect)	Rectangle
shape.draw_circle(center, radius)	Cercle
shape.draw_oval(rect)	Ellipse dans un rect
shape.draw_bezier(p1, p2, p3, p4)	Courbe de Bézier cubique
shape.draw_curve(p1, p2, p3)	Courbe quadratique
shape.draw_polyline(points)	Polyligne
shape.draw_sector(center, point, angle)	Secteur angulaire
shape.draw_squiggle(p1, p2)	Ligne ondulée
shape.finish(	
color=(1, 0, 0),	Couleur du trait (RGB 0-1)
fill=(1, 1, 0),	Couleur de remplissage
width=1.5,	Épaisseur du trait
dashes="[3 2] 0",	Pointillés (format PDF)
closePath=True,	Ferme le chemin
even_odd=False,	Règle de remplissage
morph=(pivot, matrix),	Transformation

<pre> lineCap=1, lineJoin=1, opacity=0.8,) shape.commit() </pre>	Extrémité de ligne (0=butt,1=round) Jonction (0=miter,1=round,2=bevel) Opacité (0-1) Applique sur la page
— TEXTE —	
<pre> page.insert_text(fitz.Point(72, 100), "Mon texte", fontsize=12, fontname="helv", color=(0, 0, 0), rotate=0,) </pre>	Insère du texte simple Position (x, y) Taille Police (helv, tiro, zadb, symb, cour, times) Couleur (RGB 0-1) Rotation (0, 90, 180, 270)
<pre> page.insert_textbox(rect, "Texte long qui s'adapte au rectangle", fontsize=10, fontname="helv", align=fitz.TEXT_ALIGN_LEFT, color=(0,0,0), fill=(1,1,1), border_width=1, expandtabs=4,) </pre>	Insère texte dans un rectangle Alignement (LEFT, CENTER, RIGHT, JUSTIFY) Fond Taille des tabulations
<pre> page.insert_htmlbox(rect, html_string) </pre>	Insère du HTML formaté (1.23+)
— IMAGES —	
<pre> page.insert_image(rect, filename="logo.png") page.insert_image(rect, stream=img_bytes) page.insert_image(rect, pixmap=pix) page.insert_image(rect, xref=xref) page.insert_image(rect, filename="f.png", keep_proportion=True) page.insert_image(rect, filename="f.png", overlay=True) page.insert_image(rect, filename="f.png", rotate=90) </pre>	Insère une image depuis fichier Insère depuis bytes Insère un Pixmap Réutilise une image déjà embarquée Conserve le ratio Au-dessus du contenu existant Rotation
— PDF DANS PDF —	
<pre> page.show_pdf_page(rect, src_doc, pno=0) page.show_pdf_page(rect, src_doc, pno=0, keep_proportion=True) </pre>	Insère une page PDF dans une autre

EXEMPLES :

```

# Watermark "CONFIDENTIEL" diagonal
shape = page.new_shape()
shape.insert_text(
    fitz.Point(page.rect.width/4, page.rect.height/2),
    "CONFIDENTIEL",
    fontsize=50,
    color=(0.8, 0, 0),
    rotate=45,
    opacity=0.3,
)
shape.commit()

# Rectangle avec fond coloré
shape = page.new_shape()
shape.draw_rect(fitz.Rect(50, 50, 200, 100))
shape.finish(color=(0,0,1), fill=(0.9,0.9,1), width=1)
shape.commit()

```

[doc] Fusion, découpe et transformation – insert_pdf / save

DESCRIPTION : Fusionner, découper, chiffrer et sauvegarder des documents.
 POURQUOI : Assemblage de PDF, extraction de plages, protection.

— FUSION —

```

doc1.insert_pdf(doc2)

```

Ajoute tout doc2 à la fin de doc1

```

doc1.insert_pdf(doc2, from_page=0, to_page=5) Pages 0 à 5 de doc2
doc1.insert_pdf(doc2, start_at=2)           Insère à la position 2
doc1.insert_pdf(doc2, rotate=90)           Avec rotation
doc1.insert_pdf(doc2, links=False)         Sans les liens
doc1.insert_pdf(doc2, annots=False)        Sans les annotations
doc1.insert_pdf(doc2, from_page=0,
                  to_page=-1, start_at=-1) Tout à la fin

```

— SAUVEGARDER —

```

doc.save("sortie.pdf")           Sauvegarde dans un fichier
doc.save("sortie.pdf",
        garbage=4,               Nettoyage (1-4, 4=max)
        deflate=True,           Compresses les flux
        deflate_images=True,     Compresses les images
        deflate_fonts=True,      Compresses les polices
        clean=True,             Nettoie le contenu
        pretty=False,           Format lisible (debug)
        incremental=False,       Sauvegarde incrémentale
        encryption=fitz.PDF_ENCRYPT_AES_256, Chiffrement
        owner_pw="admin",       Mot de passe propriétaire
        user_pw="user",         Mot de passe utilisateur
        permissions=0b11111100, Droits d'accès
    )

doc.tobytes()                   Sauvegarde en bytes en mémoire
doc.tobytes(garbage=4, deflate=True) Avec options
doc.write()                     Alias de tobytes()
doc.saveIncr()                  Sauvegarde incrémentale sur place
doc.ez_save("sortie.pdf")       Sauvegarde simplifiée (garbage+deflate)

```

— CHIFFREMENT ET PERMISSIONS —

```

fitz.PDF_ENCRYPT_NONE           Pas de chiffrement
fitz.PDF_ENCRYPT_RC4_40         RC4 40 bits (faible)
fitz.PDF_ENCRYPT_RC4_128        RC4 128 bits
fitz.PDF_ENCRYPT_AES_128        AES 128 bits
fitz.PDF_ENCRYPT_AES_256        AES 256 bits (recommandé)

fitz.PDF_PERM_PRINT            Permission d'imprimer
fitz.PDF_PERM_COPY             Permission de copier
fitz.PDF_PERM_ANNOTATE         Permission d'annoter
fitz.PDF_PERM_FORM              Permission de remplir les formulaires

```

— LIENS ET SIGNETS —

```

doc.get_toc()                   Table des matières [(niveau, titre, page)]
doc.set_toc(toc)                Définit la table des matières
page.get_links()                Liens de la page
page.insert_link({"kind": fitz.LINK_URI, "from": rect, "uri": "https://..."}) Lien externe
page.insert_link({"kind": fitz.LINK_GOTO, "from": rect, "page": 5}) Lien interne

```

EXEMPLES :

```

# Fusionner plusieurs PDFs
resultat = fitz.open()
for fichier in ["a.pdf", "b.pdf", "c.pdf"]:
    with fitz.open(fichier) as src:
        resultat.insert_pdf(src)
resultat.ez_save("fusion.pdf")

# Extraire une plage de pages
with fitz.open("source.pdf") as src:
    extrait = fitz.open()
    extrait.insert_pdf(src, from_page=2, to_page=7)
    extrait.save("extrait.pdf")

# Protéger un PDF
doc.save("protege.pdf",
        encryption=fitz.PDF_ENCRYPT_AES_256,
        owner_pw="admin123",
        user_pw="lecture",
        permissions=fitz.PDF_PERM_PRINT | fitz.PDF_PERM_COPY)

```


DESCRIPTION : Lire et remplir les formulaires interactifs PDF (AcroForm).
 POURQUOI : Automatiser le remplissage de formulaires, extraire les données.

FONCTIONS PRINCIPALES :

<code>page.widgets()</code>	Itère les champs du formulaire
<code>for widget in page.widgets():</code>	
<code>widget.field_name</code>	Nom du champ
<code>widget.field_value</code>	Valeur actuelle
<code>widget.field_type</code>	Type (TEXT, CHECKBOX, RADIOBUTTON...)
<code>widget.field_type_string</code>	Type en string
<code>widget.rect</code>	Position/taille
<code>widget.text_maxlen</code>	Longueur max (champs texte)
<code>widget.choice_values</code>	Options (listes déroulantes)
<code>widget.is_signed</code>	True si champ signé
<code>widget.field_flags</code>	Drapeaux du champ
 <code>widget.field_value = "Nouvelle valeur"</code>	 Modifie la valeur
<code>widget.update()</code>	Applique la modification

TYPES DE CHAMPS :

<code>fitz.PDF_WIDGET_TYPE_TEXT</code>	Champ texte
<code>fitz.PDF_WIDGET_TYPE_CHECKBOX</code>	Case à cocher
<code>fitz.PDF_WIDGET_TYPE_RADIOBUTTON</code>	Bouton radio
<code>fitz.PDF_WIDGET_TYPE_COMBOBOX</code>	Liste déroulante
<code>fitz.PDF_WIDGET_TYPE_LISTBOX</code>	Liste à sélection
<code>fitz.PDF_WIDGET_TYPE_SIGNATURE</code>	Signature

EXEMPLES :

```
with fitz.open("formulaire.pdf") as doc:
    page = doc[0]
    for widget in page.widgets():
        print(f"{widget.field_name}: {widget.field_value}")
        if widget.field_name == "nom":
            widget.field_value = "Alice Dupont"
            widget.update()
    doc.save("formulaire_rempli.pdf")
```

DESCRIPTION : Classes de géométrie utilisées partout dans PyMuPDF.
 POURQUOI : Définir zones, positions, transformations.

— RECT —

<code>fitz.Rect(x0, y0, x1, y1)</code>	Rectangle (coin haut-gauche, bas-droit)
<code>fitz.Rect(0, 0, 595, 842)</code>	Page A4 entière
<code>r.x0, r.y0, r.x1, r.y1</code>	Coordonnées
<code>r.width, r.height</code>	Dimensions
<code>r.tl, r.tr, r.bl, r.br</code>	Coins (top-left, etc.)
<code>r.is_empty</code>	True si surface nulle
<code>r.is_infinite</code>	True si infini
<code>r.contains(point_ou_rect)</code>	Test d'inclusion
<code>r.intersects(autre_rect)</code>	Test d'intersection
<code>r & autre_rect</code>	Intersection
<code>r autre_rect</code>	Union
<code>r + delta</code>	Translation par Point
<code>r.include_point(point)</code>	Étend pour inclure le point
<code>r.include_rect(rect)</code>	Étend pour inclure le rect
<code>r.normalize()</code>	Normalise ($x_0 \leq x_1$, $y_0 \leq y_1$)
<code>r.irect</code>	Rect converti en IRect (entiers)
<code>r.get_area()</code>	Surface
<code>r.morph(pivot, matrix)</code>	Transformation géométrique
<code>r.round()</code>	Arrondit au pixel
<code>r.transform(matrix)</code>	Applique une matrice

— POINT —

<code>fitz.Point(x, y)</code>	Point 2D
<code>p.x, p.y</code>	Coordonnées
<code>p + autre_point</code>	Addition
<code>p * nombre</code>	Multiplication scalaire
<code>p.distance_to(autre_point)</code>	Distance euclidienne

p.transform(matrix) Applique une matrice

— MATRIX —

fitz.Matrix(zoom_x, zoom_y)	Zoom
fitz.Matrix(1, 1)	Identité
fitz.Matrix(2, 2)	Zoom x2
fitz.Matrix(angle)	Rotation (degrés)
fitz.Matrix(zoom_x, zoom_y).prerotate(90)	Zoom + rotation
mat.a, mat.b, mat.c, mat.d, mat.e, mat.f	Composantes de la matrice 2x3
mat * autre_mat	Composition
mat.invert()	Inverse

— CONVERSIONS DPI / POINTS —

PDF utilise des points (pts) : 1 pt = 1/72 pouce
A4 : 595 x 842 pts = 21 x 29.7 cm
US Letter : 612 x 792 pts
Pour 300 DPI : zoom = 300/72 ≈ 4.17
mat = fitz.Matrix(300/72, 300/72) Rendu à 300 DPI

EXEMPLES :

```
# Zone du quart supérieur gauche
page_rect = page.rect
zone = fitz.Rect(
    page_rect.x0, page_rect.y0,
    page_rect.width / 2, page_rect.height / 2
)
texte = page.get_text(clip=zone)
```

===== RÉCAPITULATIF PYMUPDF (FITZ) =====

TÂCHE	MÉTHODE PRINCIPALE
Ouvrir un PDF	fitz.open("fichier.pdf")
Accéder à une page	doc[n] ou doc.load_page(n)
Extraire le texte	page.get_text() / page.get_text("dict")
Chercher du texte	page.search_for("terme")
Rendre en image	page.get_pixmap(dpi=150)
Extraire les images	page.get_images() + doc.extract_image(xref)
Extraire les tableaux	page.find_tables()
Surligner	page.add_highlight_annot(rect)
Caviarder	page.add_redact_annot(r) + apply_redactions()
Insérer du texte	page.insert_text(point, "texte")
Insérer une image	page.insert_image(rect, filename="f.png")
Dessiner des formes	page.new_shape() + draw_* + finish() + commit()
Fusionner des PDFs	doc1.insert_pdf(doc2)
Découper des pages	doc.insert_pdf(src, from_page, to_page)
Supprimer des pages	doc.delete_page(n)
Réordonner les pages	doc.select([...])
Sauvegarder	doc.save("out.pdf") / doc.ez_save("out.pdf")
Sauvegarder en mémoire	doc.tobytes()
Chiffrer	doc.save(..., encryption=PDF_ENCRYPT_AES_256)
Table des matières	doc.get_toc() / doc.set_toc(toc)
Formulaires	page.widgets() + widget.update()
Métadonnées	doc.metadata / doc.set_metadata(dict)
Rendre une zone	page.get_pixmap(clip=rect)

— DIMENSIONS COURANTES (points PDF) —

FORMAT	LARGEUR	HAUTEUR	ORIENTATION
A4	595.28	841.89	Portrait
A4 paysage	841.89	595.28	Paysage
A3	841.89	1190.55	Portrait
Letter (US)	612.00	792.00	Portrait
Legal (US)	612.00	1008.00	Portrait